LAB 03 CHEATSHEET — Miền Tần Số & Miền Không Gian + Mini-Project Pipeline

Bui Quang Chien • 23001837 • MAT3562E Computer Vision 2025–2026

1. Biến đổi Fourier rời rạc (DFT / FFT)

Công thức DFT 2D:

$$F(u,v) = \sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x,y)\, e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

IDFT 2D:

$$f(x,y) = \frac{1}{MN}\sum_{u}\sum_{v} F(u,v)\, e^{j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

FFT bằng NumPy

```
F  = np.fft.fft2(img)          # DFT 2D -> so phuc
Fs = np.fft.fftshift(F)        # DC ra tam
S  = np.log(1 + np.abs(Fs))    # Pho bien do (log scale)
```

Lưu ý:

- `fftshift`: dịch DC từ góc ra **chính giữa**
- Dùng `log(1+|F|)`: DC rất lớn, log chặn tương phản
- DC = giá trị trung bình tất cả pixel ⇒ điểm sáng nhất ở tâm
- Ảnh nhiều texture ⇒ phổ lan rộng ra xa tâm hơn

Độ phức tạp: DFT ngây thơ $O(M^2N^2)$; FFT $O(MN\log MN)$

2. Lọc trong miền tần số

Bộ lọc thông thấp lý tưởng (Ideal LPF)

```
def circular_lpf_mask(shape, r):
    M, N = shape
    crow, ccol = M//2, N//2
    Y, X = np.ogrid[:M, :N]
    dist2 = (Y-crow)**2 + (X-ccol)**2
    return (dist2 <= r*r).astype(np.float64)

Hlp = circular_lpf_mask(img.shape, r=30)
Hhp = 1 - Hlp  # HPF = 1 - LPF
```

Áp dụng bộ lọc

```
def apply_freq_filter(image, mask):
    F  = np.fft.fft2(image)
    Fs = np.fft.fftshift(F)
    Gs = Fs * mask                          # Nhan pho voi mat na
    g  = np.fft.ifft2(np.fft.ifftshift(Gs))
    return np.abs(g)
```

r	LPF	HPF
Nhỏ (10)	Rất mờ	Chỉ biên mạnh
Vừa (30)	Mờ vừa	Biên rõ
Lớn (60)	Gần gốc	Chi tiết nhiều

Hiệu ứng Ringing (Gibbs): Ideal LPF cắt ngưỡng đột ngột ⇒ sóng gợn quanh biên. Gaussian LPF mượt hơn, ít ringing hơn.

3. Tích chập rời rạc 2D

Công thức: $(f*h)(x,y) = \sum_i\sum_j f(x-i,y-j)\cdot h(i,j)$

Định lý tích chập: $f*h \iff F\cdot H$

```
# Tu cai dat (same padding, zero-pad)
def conv2d_same(image, kernel):
    kh, kw = kernel.shape
    ph, pw = kh//2, kw//2
    padded = np.pad(image, ((ph,ph),(pw,pw)), 'constant')
    H, W = image.shape
    out = np.zeros((H,W), dtype=np.float64)
    for i in range(H):
        for j in range(W):
            out[i,j] = np.sum(padded[i:i+kh,j:j+kw]*kernel)
    return out

# OpenCV (BORDER_REFLECT_101 mac dinh)
out_cv = cv2.filter2D(image, -1, kernel)
```

Sai khác vùng biên: zero-padding → viền tối; `cv2.filter2D` dùng reflect → tự nhiên hơn. Vùng trong: kết quả **gần như giống hệt**.

4. Các bộ lọc làm trơn

Trung bình (Mean / Box Filter)

```
k_mean = np.ones((5,5)) / 25.0   # NumPy
out_cv = cv2.blur(image, (5,5))   # OpenCV
```

Gaussian Filter

$$G(x,y) = \frac{1}{2\pi\sigma^2}e^{-\frac{x^2+y^2}{2\sigma^2}}$$

```
out = cv2.GaussianBlur(image, (ksize, ksize), sigma)
# sigma=0 -> tu tinh; sigma=0.5 -> nhe; sigma=2.0 -> mo ro
```

	Mean	Gaussian
Kernel	Đều	Giảm dần từ tâm
Giữ biên	Kém	Tốt hơn
Tăng ksize	Mờ đều, lan biên	Mờ mượt, biên mềm

5. Bộ lọc trung vị (Median)

```
out = cv2.medianBlur(image, ksize)  # ksize phai le: 3,5,7,...
```

Khi nào dùng Median? Nhiễu Salt & Pepper (nhiễu xung). Median **robust với outlier**: giá trị 0/255 luôn nằm ở 2 đầu khi sắp xếp ⇒ bị loại bỏ hoàn toàn. Mean/Gaussian chỉ **lan** nhiễu xung ra xung quanh.

6. Khử nhiễu — Chọn bộ lọc phù hợp

```
# Them nhieu Gaussian
def add_gaussian_noise(image, sigma=20):
    noise = np.random.normal(0, sigma, image.shape)
    return np.clip(image.astype(np.float64)+noise, 0,255).astype(np.
        uint8)

# Them nhieu Salt & Pepper
def add_salt_pepper(image, prob=0.02):
    out = image.copy()
    rnd = np.random.rand(*image.shape)
    out[rnd < prob/2] = 0
    out[rnd > 1-prob/2] = 255
    return out
```

Nhiễu	Bộ lọc tốt nhất	Lý do
Gaussian	GaussianBlur	Cùng phân phối
Salt & Pepper	MedianBlur	Robust với outlier
Gaussian nhẹ	Mean filter	Đơn giản, hiệu quả

7. Làm sắc nét (Sharpening)

Phương pháp 1: Laplacian

$$\nabla^2 = \begin{bmatrix}0 & 1 & 0\\ 1 & -4 & 1\\ 0 & 1 & 0\end{bmatrix}, \quad g = f - \lambda\nabla^2 f$$

```
k_lap = np.array([[0,1,0],[1,-4,1],[0,1,0]], dtype=np.float64)
lap = conv2d_same(image, k_lap)
sharp = np.clip(image.astype(np.float64) - lam*lap, 0,255).astype(np
    .uint8)
# lam=0.3: nhe; lam=0.7: vua; lam>=1.2 -> artifact ro
```

Phương pháp 2: Unsharp Masking (khuyến nghị!)

$$g = f + k\,\underbrace{(f - f_{\text{blur}})}_{\text{high-freq mask}}$$

```
blur = cv2.GaussianBlur(image, (5,5), 1.0)
hf = image.astype(np.float64) - blur.astype(np.float64)
sharp = np.clip(image.astype(np.float64) + k*hf, 0,255).astype(np.
    uint8)
# k=0.5: nhe; k=1.0: vua; k>=1.5 -> overshoot
```

	Laplacian	Unsharp Masking
Nhạy nhiễu	Rất nhạy (bậc 2)	Ít nhạy (blur lọc trước)
Artefact	Halo mạnh	Overshoot nhẹ
Dễ dùng	Cần chỉnh λ	Ít rủi ro hơn

8. Dò biên Sobel

$$G_x = \begin{bmatrix}-1 & 0 & 1\\ -2 & 0 & 2\\ -1 & 0 & 1\end{bmatrix}, \quad G_y = \begin{bmatrix}-1 & -2 & -1\\ 0 & 0 & 0\\ 1 & 2 & 1\end{bmatrix}$$

$$\|\nabla f\| = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan 2(G_y, G_x)$$

```
Gx = np.array([[-1,0,1],[-2,0,2],[-1,0,1]], dtype=np.float64)
Gy = np.array([[-1,-2,-1],[0,0,0],[1,2,1]], dtype=np.float64)
Sx = conv2d_same(img, Gx);  Sy = conv2d_same(img, Gy)
mag = np.sqrt(Sx**2 + Sy**2)
# Nguong hoa
edge_map = (mag >= T).astype(np.uint8) * 255
# T=30: nhieu bien; T=60: can bang; T=90: sach but dut
```

5 bước Canny:

1. **Gaussian blur** — loại nhiễu
2. **Sobel gradient** — cường độ + hướng biên
3. **NMS** — làm mảnh biên (chỉ giữ cực đại)
4. **Double threshold** — phân loại strong/weak
5. **Hysteresis** — kết nối weak → strong

```
edges = cv2.Canny(image, threshold1=30, threshold2=80)
```

NMS (Non-Maximum Suppression)

```python
def non_maximum_suppression(G, theta):
    angle = theta * 180/np.pi
    angle[angle < 0] += 180
    Z = np.zeros_like(G)
    for i in range(1, G.shape[0]-1):
        for j in range(1, G.shape[1]-1):
            if (0<=angle[i,j]<22.5) or (157.5<=angle[i,j]<=180):
                q,r = G[i,j+1], G[i,j-1]   # 0 do
            elif 22.5<=angle[i,j]<67.5:
                q,r = G[i+1,j-1], G[i-1,j+1] # 45 do
            elif 67.5<=angle[i,j]<112.5:
                q,r = G[i+1,j], G[i-1,j]   # 90 do
            else:
                q,r = G[i-1,j-1], G[i+1,j+1] # 135 do
            if G[i,j]>=q and G[i,j]>=r:
                Z[i,j] = G[i,j]
    return Z
```

Double Threshold + Hysteresis

```python
def double_threshold(img, t_low, t_high, weak=50, strong=255):
    res = np.zeros_like(img, dtype=np.uint8)
    res[img >= t_high] = strong
    res[(img >= t_low) & (img < t_high)] = weak
    return res

def hysteresis(img, weak=50, strong=255):
    out = img.copy()
    for i in range(1, out.shape[0]-1):
        for j in range(1, out.shape[1]-1):
            if out[i,j] == weak:
                out[i,j] = strong if np.any(
                    out[i-1:i+2,j-1:j+2]==strong) else 0
    return out
```

(T_l, T_h)	Biên	Nhiễu
$(20, 60)$	Nhiều liên tục	Nhiễu nhiều
$(30, 80)$	**Cân bằng**	Vừa phải
$(50, 120)$	Ít, sạch	Ít, nhưng đứt

Vì sao hysteresis tốt hơn ngưỡng đơn?
Giữ weak edge **chỉ khi** nó kết nối với strong edge ⇒ loại nhiễu cô lập, biên liên tục hơn.

10. So sánh Spatial vs Frequency Filtering

	Spatial	Frequency
Thao tác	Kernel trượt	Nhân phổ
Gaussian blur	GaussianBlur	Gaussian LPF
Giống nhau?	Không hoàn toàn	(ideal ≠ Gaussian)
Trực quan	Dễ	Cần biết FFT
Ứng dụng	Thông dụng	Lọc chọn lọc

Gaussian spatial blur ≠ Circular LPF vì:
Circular LPF lý tưởng cắt ngưỡng đột ngột ⇒ ringing. Gaussian blur ↔ Gaussian LPF (mượt, không ringing).

11. Mini-Project: Pipeline hoàn chỉnh

Ảnh gốc → Nhiễu → Khử nhiễu → Sắc nét → Biên

```python
img_clean = img_edge.copy()   # anh sach ban dau

# === Pipeline 1: Gaussian noise ===
noisy_g = add_gaussian_noise(img_clean, sigma=25)
denoised_g = cv2.GaussianBlur(noisy_g, (5,5), 1.0)
blur_tmp = cv2.GaussianBlur(denoised_g, (5,5), 1.0)
hf = denoised_g.astype(np.float64) - blur_tmp.astype(np.float64)
sharp_g = np.clip(denoised_g.astype(np.float64)+1.0*hf, 0,255).
      astype(np.uint8)
edges_g = cv2.Canny(sharp_g, 30, 80)

# === Pipeline 2: Salt & Pepper noise ===
noisy_sp = add_salt_pepper(img_clean, prob=0.03)
denoised_sp = cv2.medianBlur(noisy_sp, 5)
blur_tmp2 = cv2.GaussianBlur(denoised_sp, (5,5), 1.0)
hf2 = denoised_sp.astype(np.float64) - blur_tmp2.astype(np.float64)
sharp_sp = np.clip(denoised_sp.astype(np.float64)+1.0*hf2, 0,255).
      astype(np.uint8)
edges_sp = cv2.Canny(sharp_sp, 30, 80)
```

Nhiễu	Khử	Sắc nét	Biên
Gaussian σ=25	GaussianBlur	Unsharp k=1	Canny(30,80)
S&P p=0.03	MedianBlur	Unsharp k=1	Canny(30,80)

Trade-off 3 chiều — Quy tắc vàng:

- Khử nhiễu mạnh ⇒ ít nhiễu nhưng **mất biên yếu**
- Sắc nét mạnh ⇒ rõ biên nhưng **khuếch đại nhiễu**
- Ngưỡng Canny cao ⇒ sạch nhưng **biên đứt đoạn**

⇒ **Không có tham số "tốt nhất" chung**, phải cân bằng cả 3 cho từng ảnh!

12. Bảng hàm OpenCV nhanh

Hàm	Mục đích
`cv2.blur(img,(k,k))`	Lọc trung bình
`cv2.GaussianBlur(img,(k,k),s)`	Lọc Gaussian
`cv2.medianBlur(img,k)`	Lọc trung vị
`cv2.filter2D(img,-1,kernel)`	Tích chập tổng quát
`cv2.Canny(img,t1,t2)`	Dò biên Canny
`np.fft.fft2(img)`	FFT 2D (complex)
`np.fft.fftshift(F)`	DC ra tâm
`np.fft.ifft2(F)`	IFFT 2D
`np.fft.ifftshift(F)`	Đảo fftshift

13. Công thức cần nhớ & Tips

Tên	Công thức
DFT	$F(u,v) = \sum_x \sum_y f\, e^{-j2\pi(ux/M+vy/N)}$
Tích chập	$(f*h)(x,y) = \sum_i \sum_j f(x-i, y-j)h(i,j)$
Conv. theorem	$f*h \Leftrightarrow F \cdot H$
Gaussian	$G = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$
Gradient	$\lVert\nabla f\rVert = \sqrt{G_x^2 + G_y^2}$
Unsharp	$g = f + k(f - f_{\text{blur}})$
Laplacian	$g = f - \lambda\nabla^2 f$

7 tips thực hành:

1. Luôn chuyển `uint8` → `float64` trước khi tính
2. `np.clip(result,0,255).astype(np.uint8)` tránh tràn số
3. Phổ FFT dùng `log(1+|F|)`, không để DC nuốt hết
4. Zero-pad → viền tối; reflect-pad → tự nhiên hơn
5. So sánh NumPy vs OpenCV: `np.abs(a-b)` diff map
6. **Khử nhiễu TRƯỚC khi dò biên** — không bao giờ ngược
7. **Median cho S&P, Gaussian cho Gaussian noise** — quy tắc vàng